JAVA LAB WEEK 14

Title: Collection Classes – Valid Parentheses Using Stack

1. Objective / Aim

To write and execute a Java program to determine whether a given string containing parentheses, curly brackets, and square brackets is valid. The program uses the Stack collection class to check whether all opening brackets are closed by the same type of closing brackets in the correct order.

2. System Requirements

Hardware Requirements:

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard and monitor

Software Requirements:

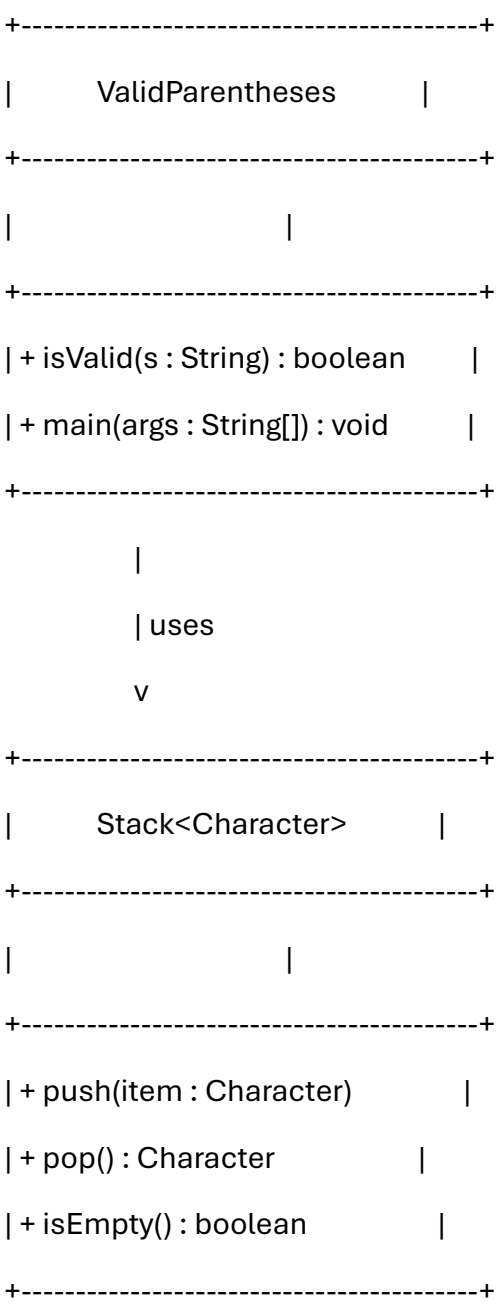
- Operating System: Windows, Linux, or macOS
- Java Development Kit (JDK): JDK 8 or above
- IDE: Eclipse, IntelliJ IDEA, NetBeans, or Visual Studio Code
- Command Prompt or Terminal

3. Algorithm / Logic Description

1. Start the program.
2. Import the java.util package to use Scanner and Stack.
3. Create a class named ValidParentheses.
4. Define a method named isValid() that accepts a string as input.
5. Create a Stack object to store opening brackets.
6. Read each character of the input string using a loop.
7. If the character is an opening bracket (, {, or [, push it onto the stack.
8. If the character is a closing bracket, check whether the stack is empty.
9. If the stack is empty, return false because there is no matching opening bracket.
10. Otherwise, remove the top opening bracket using the pop() method.
11. Check whether the opening bracket matches the closing bracket.

12. If the brackets do not match, return false.
13. After checking all characters, verify whether the stack is empty.
14. If the stack is empty, return true; otherwise, return false.
15. In the main() method, read the bracket string from the user.
16. Call the isValid() method and display Valid or Not valid.
17. Close the Scanner object.
18. Stop the program.

4. UML Class Diagram



Description:

- ValidParentheses is the main user-defined class.
- isValid(String s) checks whether the given bracket string is valid.
- main(String[] args) is the entry point of the program.
- Stack<Character> stores the opening brackets.
- push() inserts an opening bracket into the stack.
- pop() removes and returns the most recently added opening bracket.
- isEmpty() checks whether the stack contains any elements.
- Scanner is used to accept input from the user.
- The program uses the Last In, First Out (LIFO) principle of the stack.

5. Source Code

```
import java.util.*;


public class ValidParentheses {


  public static boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();


    for (int i = 0; i < s.length(); i++) {
      char ch = s.charAt(i);


      if (ch == '(' || ch == '{' || ch == '[') {
        stack.push(ch);
      } else {
        if (stack.isEmpty()) {
          return false;
        }
```

```
            char top = stack.pop();

            if (ch == ')' && top != '(') {
                return false;
            }

            if (ch == '}' && top != '{') {
                return false;
            }

            if (ch == ']' && top != '[') {
                return false;
            }
        }
    }

    return stack.isEmpty();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter brackets: ");
    String input = sc.nextLine();

    if (isValid(input)) {
        System.out.println("Valid");
    } else {
```

```
            System.out.println("Not valid");
        }


        sc.close();
    }
}
```

6. Compilation and Execution Commands

Step 1: Save the program

Save the Java program with the filename ValidParentheses.java.

Step 2: Open Command Prompt or Terminal

Navigate to the folder where the Java program is saved.

Step 3: Compile the program

javac ValidParentheses.java

Step 4: Execute the program

java ValidParentheses

7. Output Screen

Example 1: Valid Parentheses

Enter brackets: ()

Valid

Explanation:

The opening bracket (is matched with the closing bracket). Therefore, the input is valid.

Example 2: Invalid Parentheses

Enter brackets: ({)}

Not valid

Explanation:

The opening bracket { must be closed before the opening bracket (can be closed. The given input has an incorrect bracket order. Therefore, the input is not valid.

Example 3: Empty String

Enter brackets:

Valid

Explanation:

An empty string contains no unmatched brackets. Therefore, it is considered valid.

Example 4: Mixed Valid Parentheses

Enter brackets: ({[]})

Valid

Explanation:

All opening brackets are closed by their corresponding closing brackets in the correct order. Therefore, the input is valid.

Important Concepts Used:

1. **Collection Framework:** Used to manage a collection of opening brackets.
2. **Stack Class:** Stores opening brackets and checks them in reverse order.
3. **LIFO Principle:** The last opening bracket must be closed first.
4. **push():** Adds an opening bracket to the stack.
5. **pop():** Removes the top element from the stack.
6. **isEmpty():** Checks whether the stack is empty.
7. **Boolean Method:** Returns true for valid input and false for invalid input.
8. **Scanner:** Accepts the bracket string from the user.

8. Result

Thus, the Java program to check whether a string contains valid parentheses was successfully implemented and executed. The experiment demonstrates the use of the Java Collection Framework, the Stack class, the LIFO principle, conditional statements, loops, and methods to validate brackets in the correct order.